

Reflection Question/Answers.

1. Which optimization produced the greatest improvement?

Among all the optimizations, connection management & improved thread handling produced the greatest improvement. Previously, disconnected clients sometimes remained active & unnecessary threads continued consuming system resources. After implementing automatic client removal, proper resource cleanup, timeout handling & efficient thread management, the server became much more stable. The application successfully supported 10 concurrent clients without crashes or noticeable performance degradation. CPU & memory utilization also became more consistent during testing. These improvements reduced communication delays, prevented resource leakage & increased overall reliability.

2. Why is Scalability Important?

Scalability is important because it allows an application to support more users without reducing performance. In a chat application, many clients may connect simultaneously, making efficient resource management essential. A scalable system maintains low response time, better throughput & stable performance even under heavy load. It also makes future expansion easier & ensures reliable communication as the ~~more~~ number of users increases.

3. what reliability issues did you discover?

During testing, I found that disconnected clients sometimes remained active, network failures interrupted communication & improper exception handling caused connection issues. Some resources were not released correctly after disconnection. I resolved these problems by implementing automatic reconnection, timeout handling, graceful shutdown & proper socket cleanup. These improvements made the application more stable, reliable & capable of recovering from unexpected network failures.

Date _____
Page _____

4. Which optimization was the most difficult?

The most difficult optimization was implementing automatic reconnection while maintaining stable communication. The application had to detect connection failures, reconnect safely, & avoid duplicate sessions.

Managing multiple threads & handling exceptions correctly required careful coding & repeated testing. After resolving these challenges, the application became more fault-tolerant & provided a smoother user experience during client disconnections.

5. What additional improvements are required to support 100 users?

To support 100 users, the application should use asynchronous I/O on a thread pool instead of one thread per client. A database should manage user information & chat history efficiently. Load balancing, message queues, caching & stronger security measures such as SSL/TLS should also be implemented. These improvements would increase Scalability, reliability, & overall System performance.